COURSE 2A

C2-06 DATA WAREHOUSING & PLATFORM FOUNDATIONS

BEGINNER-FIRST TEACHING BOOK - RC1

OLTP/OLAP -> warehouse -> dimensional history -> layers -> lakehouse -> quality

THIS BOOK IS YOUR TEACHER

Very High DE bridge. Vendor-neutral concepts first; Fabric used only as current implementation example.

This book is your teacher

Architecture is learned by drawing grain, history, layers and controls - not memorizing platform nouns

THE STUDY LOOP

Why -> mental picture -> vocabulary -> follow one small architecture -> expected result -> why it works -> common mistakes -> recovery -> fresh design task -> validate -> explain-back -> save attempt -> protected review -> changed retry.

NO CLOUD-TENANT DEPENDENCY

Core mastery is architecture/design evidence using the supplied workbook and datasets. Fabric links illustrate current implementation patterns. A paid/trial Fabric workspace is not required to learn or pass the conceptual design Gate.

WHAT IS CURRENT

Microsoft's 2026 Fabric guidance recommends medallion lakehouse organization and provides current Warehouse/Lakehouse data-store decision guidance. Those product details may change, so the durable internal principles remain authoritative.

WHAT ADVANCED MEANS

You can defend workload placement, fact grain, key/history design, ETL/ELT/layers, warehouse-vs-lakehouse choice, latency, lineage and a multi-stage quality plan. You are not expected to build a production Fabric platform here.

Contents / Index

Every canonical DWF lesson and actual page number

Prerequisite and tool map

What is assumed, what is new and what belongs to C2-07 instead

ASSUMED

Course 1 SQL/Power BI basics; C2-00 grain/validation language; C2-03 fact/dimension/filter thinking. C2-05 reliability habits help but are not required. New warehouse/history/platform concepts are taught from first principles.

TOOLS

Primary work is design/QA in the provided workbook and CSV evidence. Optional SQL/Power BI may be used to validate familiar concepts. Fabric workspace execution is not required in C2-06; current Microsoft Learn visuals are enough for product examples.

NEW DEPTH

Operational vs analytical workloads; warehouse purpose; dimensional grain; surrogate keys; SCD history; ETL/ELT; layers/replay; lake/lakehouse/store decisions; batch/streaming latency; lineage/metadata; multi-stage quality gates.

C2-07 BOUNDARY

C2-06 teaches portable architecture reasoning. C2-07 owns deeper Cloud/Fabric ecosystem, compute/storage/identity, workspaces/deployment, gateways, permissions/governance and monitoring. Do not duplicate C2-07 here.

Practice, review and Gate route

Architecture evidence must be explicit and transferable

LESSON RULE

Use the matching P-DWF workbook sheet only after reading the lesson. Save the first design, assumptions and independent validation before opening protected review. Close review before changed transfer.

MINI-LAB

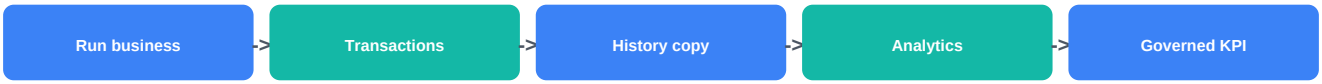
FleetCare cumulative warehouse design integrates Sales/Support/Targets, conformed dimensions, Customer SCD2, layers, ETL/ELT, store/latency, lineage and 8+ quality controls.

GATES

A Logistics Shipments, B E-commerce Returns, C Clinic Appointments. Distinct processes/grains/history defects/target grains. Any critical grain/history/FK/reconciliation flaw blocks pass.

DWF1 - Operational databases vs analytical systems

Page A - why, outcome, prerequisite, mental model and vocabulary



WHY IT MATTERS

The database that records an order correctly is solving a different job from the system that analyzes eighteen months of orders. Running heavy analytical joins on operational systems can hurt the business and still fail to preserve historical meaning.

WHAT YOU LEARN

Classify systems by workload instead of product name; separate transaction integrity from analytical history; explain why analytical copies need freshness, lineage and governed definitions.

PREREQUISITE

Course 1 database basics and C2-02/C2-03 grain concepts. No engineering platform experience required.

NEW WORDS BEFORE WE USE THEM

Operational system: System primarily designed to record/run day-to-day business transactions. Analytical system: System designed for historical scans, aggregations, cross-source analysis and governed reporting. Workload isolation: Keeping heavy analytics from competing directly with critical transaction processing.

DWF1 - Follow with me once

Page B - Order application versus sales-history platform

Order application versus sales-history platform

1. Operational question: Did order O1023 post, reserve inventory and record payment correctly? This needs current transactional correctness.
2. Analytical question: Which customer segments drove revenue growth by region over 18 months? This needs history, broad scans and stable dimensions.
3. If Customer.Region changes today, the operational system may simply store the current region. Historical reporting may need the region that was valid when each sale occurred.
4. Sketch Source OLTP -> governed analytical copy -> semantic/report layer. Note that the copied data is now subject to freshness and lineage controls.
5. Write one reason not to run a large executive-history query directly against the production order database.

EXPECTED RESULT

You can classify a workload as operational or analytical and name the history/freshness controls that appear after data is copied.

WHY IT WORKED

The distinction comes from the job the system performs, not from calling one database 'modern' and another 'old'.

DWF1 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: an insurance claims app and an executive loss-analysis platform. Classify five example queries/actions and justify placement.

COMMON BEGINNER MISTAKES

1. Calling any SQL database OLTP and any warehouse OLAP. | 2. Assuming operational current-state rows preserve historical attribution. | 3. Ignoring source-system load/availability when analysts query production. | 4. Treating copied data as automatically current/trusted.

STUCK? RECOVERY

Write the user action first: record a transaction or analyze many historical records? Then ask latency, concurrency, history and scan pattern. Save the genuine attempt before review.

VALIDATE

For each case record workload type, history need, latency and one risk if run in the wrong system.

DWF1 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Architectures often include operational sources plus analytical stores, replicas or lake/warehouse layers. The right boundary protects transaction systems while giving analysts stable historical data.

TEACH IT BACK

Explain operational versus analytical systems using 'run the business' versus 'understand the business' without saying one is better.

PROTECTED REVIEW + CHANGED RETRY

Save classification first. Changed retry uses hospital appointment booking versus care-capacity analytics.

EVIDENCE / MASTERY

Workload table + architecture sketch + risk/freshness controls + explain-back.

VISUAL/SOURCE ROUTE: Microsoft Fabric warehouse module: Understand data warehouses (7 min) is recommended visual context.

https://learn.microsoft.com/en-us/training/modules/get-started-data-warehouse/

DWF2 - OLTP vs OLAP

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

OLTP and OLAP are workload patterns, not insults about schema quality. A normalized transactional schema can be excellent for updates while inconvenient for analytical questions.

WHAT YOU LEARN

Compare OLTP and OLAP on transaction size, concurrency, update patterns, history, query shapes and common schema design; recognize that analytical systems can still be near-real-time.

PREREQUISITE

DWF1 workload separation and basic relational database concepts.

NEW WORDS BEFORE WE USE THEM

OLTP: Online Transaction Processing: many small operational reads/writes with transactional integrity. OLAP: Online Analytical Processing: analytical scans/aggregations over larger/historical datasets. Normalization: Structuring relational data to reduce redundancy and protect update integrity.

DWF2 - Follow with me once

Page B - Two questions over the same business

Two questions over the same business

1. OLTP: 'Did customer C12's payment complete?' likely touches a few rows and needs fast, correct transaction handling.
2. OLAP: 'Which segments/regions drove margin change by quarter?' scans/aggregates many records and may join historical dimensions.
3. Explain why normalized Customer, Order, OrderLine, Payment tables can be right for transactions.
4. Explain why an analytical dimensional model may denormalize descriptive attributes around facts for simpler reporting.
5. Record that OLAP does not require stale data: near-real-time analytical copies can exist if the business needs that latency.

EXPECTED RESULT

A workload comparison based on actual behavior, with no claim that normalization is bad design.

WHY IT WORKED

The same business can need both structures because transaction writes and analytical scans optimize different outcomes.

DWF2 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: food-delivery order placement, courier tracking, daily finance close and monthly cohort analysis. Classify OLTP/OLAP/near-real-time analytical needs.

COMMON BEGINNER MISTAKES

1. OLAP = old data only. | 2. Denormalized = automatically faster/correct. | 3. OLTP schema is 'wrong' because BI queries are verbose. | 4. Choosing architecture before defining query/latency/concurrency needs.

STUCK? RECOVERY

For each workload write: writes per second, rows touched, history span, common query shape, acceptable latency and consistency need. Save the genuine attempt before review.

VALIDATE

Each classification cites workload properties, not a vendor/product label.

DWF2 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Hybrid and near-real-time patterns blur strict boundaries, but the workload questions remain useful: transaction integrity, scan volume, freshness, governance and cost.

TEACH IT BACK

Explain why the same company normally has both OLTP and OLAP workloads.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry classifies banking transfers and regulatory analytics.

EVIDENCE / MASTERY

OLTP/OLAP matrix + workload reasoning + explain-back.

VISUAL/SOURCE ROUTE: Book-led; Microsoft warehouse module reinforces warehouse workload purpose.

https://learn.microsoft.com/en-us/training/modules/get-started-data-warehouse/

DWF3 - Data warehouses

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

A warehouse is not just a large database. It is a governed analytical system that integrates sources, preserves useful history and serves stable business structures.

WHAT YOU LEARN

Describe warehouse purpose, subject areas, load/ownership responsibilities, historical behavior, dimensional serving and current Fabric Warehouse as one implementation example.

PREREQUISITE

DWF1-DWF2 and basic SQL/BI concepts.

NEW WORDS BEFORE WE USE THEM

Data warehouse: Governed analytical store integrating business data for historical analysis/reporting. Subject area: Business domain organized for analysis, such as Sales, Finance or Customer Service. Data mart: Curated analytical subset focused on a business area/audience.

DWF3 - Follow with me once

Page B - From exports to governed sales warehouse

From exports to governed sales warehouse

Sources: order system, CRM/customer master and targets. List owners and expected arrival/freshness.

Landing/staging captures source data and validates schema/types before business modeling.

Warehouse/core contains conformed DimCustomer, DimProduct, DimDate plus FactSales and FactTarget at declared grains.

Semantic/report layer uses those governed structures instead of rejoining raw exports monthly.

Define ownership for definitions, loads, quality, retention and failure response. A warehouse without operations/ownership is incomplete.

EXPECTED RESULT

A warehouse architecture tied to business processes, history and ownership rather than a copy of every source table.

WHY IT WORKED

The design creates a controlled analytical contract between changing operational sources and stable reporting.

DWF3 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: design a high-level warehouse for Sales + Support + Targets without choosing a cloud vendor.

COMMON BEGINNER MISTAKES

1. Copy every source table and call it a warehouse. | 2. Start with tables before business processes/consumers. | 3. No owner for failed refresh or KPI definition. | 4. Assume today's dimension value should rewrite yesterday's facts.

STUCK? RECOVERY

Start with business questions/processes, then facts/grains, dimensions/history, sources/loads, quality and consumers. Save the genuine attempt before review.

VALIDATE

Every proposed table/layer has a purpose, grain/role, upstream source and downstream consumer.

DWF3 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Fabric Warehouse is a current SaaS relational warehouse on OneLake/Delta with T-SQL and enterprise BI use cases, but durable warehouse design does not depend on Fabric.

TEACH IT BACK

Explain the difference between 'a database containing lots of data' and a governed warehouse.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses healthcare operations.

EVIDENCE / MASTERY

Architecture + subject/process map + ownership/freshness notes + explain-back.

VISUAL/SOURCE ROUTE: Microsoft Learn - Get started with data warehouses in Fabric (1h16m).

Required: Understand data warehouses 7m + warehouses in Fabric 8m + modeling 7m.

https://learn.microsoft.com/en-us/training/modules/get-started-data-warehouse/

DWF4 - Facts, dimensions and dimensional modeling

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Mixing Sales, Tickets and Targets because they share Customer or Region produces grain errors. Dimensional modeling starts with the measurable business process and one-row meaning.

WHAT YOU LEARN

Declare fact grain first; choose fact measures/foreign keys; design dimensions/conformed dimensions; keep different processes/grains separate; sketch a star schema.

PREREQUISITE

C2-03 modeling helps, but warehouse-side dimensional modeling is taught again from first principles.

NEW WORDS BEFORE WE USE THEM

Fact: Table storing measurable business events/snapshots at explicit grain. Dimension: Descriptive entity/context used to filter, group and label facts. Conformed dimension: Consistently defined dimension reused across multiple facts.

DWF4 - Follow with me once

Page B - Sales, Returns and Monthly Targets

Sales, Returns and Monthly Targets

1. FactSales grain = one row per order line. Measures may include Quantity, GrossSales, Discount, NetSales.
2. FactReturns grain = one row per return event/item. Keep it separate from sales if the process/grain differs.
3. FactTarget grain = one row per Region-Month. Do not join target value directly to every order line.
4. Conformed DimDate/DimProduct/DimCustomer/DimRegion can filter compatible facts consistently.
5. Validate each fact's business key uniqueness expectations and show which dimensions apply at its grain.

EXPECTED RESULT

A star-schema design where every fact has a written grain and higher-grain facts do not multiply through transaction rows.

WHY IT WORKED

Processes remain separate while shared dimensions provide consistent analysis.

DWF4 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: parcel Shipments, ShipmentEvents and HubMonthlyTargets with Customer/Hub/Date dimensions.

COMMON BEGINNER MISTAKES

1. Choose columns before declaring grain. | 2. One giant fact for unrelated processes. | 3. Treat target table as a dimension. | 4. Fact-to-fact joins because both have CustomerID.

STUCK? RECOVERY

Write 'one row in this fact means...' for every measurable table. Separate facts whenever that sentence differs materially. Save the genuine attempt before review.

VALIDATE

Trace five analytical questions to a fact at the right grain; prove target cannot duplicate through shipment rows.

DWF4 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Fabric's dimensional-model guidance recommends star schemas for warehouse analytics, but grain/conformed-dimension principles transfer to any warehouse platform.

TEACH IT BACK

Explain why two tables that both contain CustomerID still may belong to separate facts.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses clinic encounters, billing and staffing targets.

EVIDENCE / MASTERY

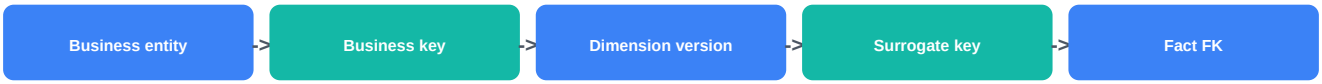
Fact-grain register + star sketch + conformed-dimension map + validation + explain-back.

VISUAL/SOURCE ROUTE: Microsoft Fabric dimensional modeling overview + optional star-schema visual from C2-03.

https://learn.microsoft.com/en-us/fabric/data-warehouse/dimensional-modeling-overview

DWF5 - Surrogate keys and business keys

Page A - why, outcome, prerequisite, mental model and vocabulary



WHY IT MATTERS

If a source business key is the only warehouse key, historical versions of the same customer/product cannot be represented cleanly. If you invent surrogate keys without preserving the business key, lineage disappears.

WHAT YOU LEARN

Distinguish source/domain business keys from warehouse-generated surrogate keys; use surrogate keys to identify dimension records/versions while preserving business keys for lineage/change detection.

PREREQUISITE

DWF4 dimensions/facts and basic primary/foreign-key concepts.

NEW WORDS BEFORE WE USE THEM

Business key: Identifier from the source/business domain for a real entity. Surrogate key: Warehouse-generated key identifying a dimension record/version. Foreign key: Key stored on a fact to reference the appropriate dimension row/version.

DWF5 - Follow with me once

Page B - Customer C005 moves regions

Customer C005 moves regions

1. Business key CustomerID = C005 remains the identity of the same real customer.
2. Warehouse row 101 may represent C005 / West / old effective period.
3. Warehouse row 155 may represent C005 / South / current period. CustomerKey differs because the dimension version differs.
4. FactSales stores the surrogate CustomerKey appropriate to the sale's historical attribution, while CustomerID remains in dimension for lineage/change detection.
5. Validate no two dimension rows share a surrogate key and each business key's historical rows obey the intended current/effective-date rules.

EXPECTED RESULT

A clear key strategy explaining real-world identity versus warehouse-record identity.

WHY IT WORKED

The warehouse can keep multiple versions without pretending the customer itself has multiple source identities.

DWF5 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: Employee E104 changes department twice. Design DimEmployee key columns and fact linkage for historical labor cost.

COMMON BEGINNER MISTAKES

1. Surrogate key replaces/erases business key. | 2. Hash/random key chosen without governance just because it sounds advanced. | 3. Facts link to current dimension version regardless of event date. | 4. Business key assumed unique across multiple sources without source/system qualification.

STUCK? RECOVERY

Ask: Which key did the source/business give us? Which key must uniquely identify a warehouse row/version? Keep both roles explicit. Save the genuine attempt before review.

VALIDATE

Surrogate uniqueness, business-key preservation and fact-to-correct-version logic are testable.

DWF5 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Surrogate keys are especially useful for SCD2 and multi-source conformance. Business keys may require source-system namespace or matching logic in real warehouses.

TEACH IT BACK

Explain business key versus surrogate key without saying 'natural keys are bad'.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses Product SKU reused across source systems.

EVIDENCE / MASTERY

Key register + sample history rows + fact foreign-key mapping + explain-back.

VISUAL/SOURCE ROUTE: Book-led; DWF6 visual makes the key/history relationship concrete.

https://learn.microsoft.com/en-us/fabric/data-warehouse/dimensional-modeling-dimension-tables

DWF6 - Slowly changing dimensions

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

If a customer moves from West to South and you overwrite Region, last year's sales can suddenly appear under today's region. SCD rules decide whether dimension changes rewrite history or preserve it.

WHAT YOU LEARN

Choose SCD behavior by attribute/business need; contrast Type 1 overwrite and Type 2 versioning; design SCD2 effective dates/current flag/surrogate keys; detect overlaps/gaps/current-row errors.

PREREQUISITE

DWF5 business/surrogate keys and DWF4 fact grain.

NEW WORDS BEFORE WE USE THEM

SCD: Slowly Changing Dimension: pattern for handling changes to descriptive dimension attributes. Type 1: Overwrite current value; prior value not preserved in that dimension history. Type 2: Insert a new dimension version and expire the old version to preserve historical attribution.

DWF6 - Follow with me once

Page B - Region changes with SCD2

Region changes with SCD2

1. Current row: CustomerKey 101, CustomerID C005, Region West, EffectiveFrom 2025-01-01, EffectiveTo open-ended, IsCurrent true.
2. Source says Region South effective 2026-04-15. Decide Region needs Type 2 history; perhaps spelling correction of CustomerName could be Type 1.
3. Expire old row at the agreed boundary and set IsCurrent false.
4. Insert new surrogate-key row for C005/South effective at change time with IsCurrent true.
5. For a sale on 2026-03-01 use old version; for 2026-05-01 use new version. Validate no overlapping effective ranges and exactly one current row per active business key.

EXPECTED RESULT

A dimension history table that preserves the approved historical meaning without overlapping versions.

WHY IT WORKED

The business key stays C005 while surrogate keys distinguish versions.

DWF6 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: Agent Team and JobTitle change on different dates; decide Type 1/2 per attribute and build valid SCD2 history.

COMMON BEGINNER MISTAKES

1. Type 2 every attribute automatically. | 2. Two current rows for one business key. | 3. Effective ranges overlap by a day/instant. | 4. Fact load joins only on business key and always receives current surrogate key.

STUCK? RECOVERY

Write the historical reporting question for each changed attribute. If prior state matters, design versioning; then draw effective-time intervals before coding. Save the genuine attempt before review.

VALIDATE

One current row per business key; no overlaps; each event date resolves to exactly one intended dimension version.

DWF6 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Current Fabric Data Factory documentation includes concrete Type 1 and Type 2 implementation patterns, but this lesson assesses vendor-neutral history design rather than copying one tool flow.

TEACH IT BACK

Explain how Type 2 prevents today's customer region from rewriting yesterday's revenue.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses Product Category history.

EVIDENCE / MASTERY

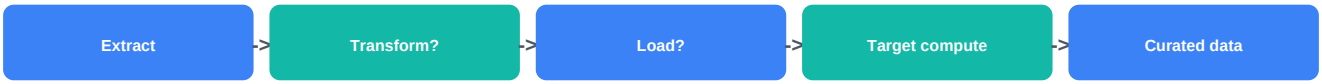
Attribute SCD policy + history rows + overlap/current tests + event-to-version mapping + explain-back.

VISUAL/SOURCE ROUTE: Recommended: Microsoft Fabric SCD Type 2 tutorial/article visuals; optional external SCD walkthrough only if it clarifies version rows.

https://learn.microsoft.com/en-us/fabric/data-factory/slowly-changing-dimension-type-two

DWF7 - ETL vs ELT

Page A - why, outcome, prerequisite, mental model and vocabulary



WHY IT MATTERS

People use ETL and ELT as labels for entire architectures even when they have not defined where transformation actually occurs. The durable question is where each validation/transformation runs and what raw history is retained.

WHAT YOU LEARN

Contrast transform-before-target ETL with load-before-transform ELT; document landing, execution location, compute/governance/security trade-offs and replay strategy.

PREREQUISITE

DWF3 warehouse architecture and PYA/C2-01 transformation concepts.

NEW WORDS BEFORE WE USE THEM

ETL: Extract, Transform, Load: transform before loading the final analytical destination. ELT: Extract, Load, Transform: land/load source data first, then transform in the target analytical platform. Replay: Reprocessing from retained earlier/raw data after logic or downstream state changes.

DWF7 - Follow with me once

Page B - Same source, two valid patterns

Same source, two valid patterns

1. ETL: source CSV -> transformation engine validates/cleans -> warehouse fact/dim tables. Raw retention is a separate design decision.
2. ELT: source CSV -> raw/landing area -> warehouse/lakehouse transformations -> curated dimensional tables.
3. Ask whether target compute can efficiently perform transformations and whether raw landing is allowed/governed.
4. Document where schema/type/quality checks run; do not hide them behind the three-letter label.
5. Plan replay: if business logic changes next month, from which trustworthy retained layer can you rebuild downstream data?

EXPECTED RESULT

An architecture that names transformation locations and retention/replay behavior rather than merely saying ETL or ELT.

WHY IT WORKED

The label is derived from actual data movement/transformation placement.

DWF7 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: CRM + finance CSV + event JSON into a governed analytical platform. Compare one ETL and one ELT design.

COMMON BEGINNER MISTAKES

1. Every cloud pipeline = ELT. | 2. ELT automatically cheaper/better. | 3. Raw landing retained forever without governance. | 4. No replay plan after transformation bug.

STUCK? RECOVERY

Draw arrows and write 'transform happens here' on every major step. Then label ETL/ELT only after the drawing makes sense. Save the genuine attempt before review.

VALIDATE

For each design record transform compute, raw retention, quality checkpoints, replay point, security and owner.

DWF7 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Modern platforms commonly support both patterns. Choose based on data size/type, target capabilities, team skills, governance, security, cost and recovery needs.

TEACH IT BACK

Explain ETL vs ELT using order of transformation and load, not cloud/on-prem marketing.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses sensitive HR data where raw landing has stricter constraints.

EVIDENCE / MASTERY

Two flow diagrams + decision matrix + replay/quality notes + explain-back.

VISUAL/SOURCE ROUTE: Microsoft Fabric warehouse learning path's load-data module is recommended implementation context; lesson remains vendor-neutral.

https://learn.microsoft.com/en-us/training/paths/work-with-data-warehouses-using-microsoft-fabric/

DWF8 - Staging, transformation and presentation layers

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

If dashboards depend directly on unstable landing files, source drift immediately becomes a reporting incident. Layers give each step a contract and allow failures to be isolated/reprocessed.

WHAT YOU LEARN

Define landing/raw, staging/standardized, core/conformed-history and presentation/consumption responsibilities; map grain, allowed changes, retention, owner and quality checks per layer.

PREREQUISITE

DWF7 ETL/ELT and DWF3 warehouse architecture.

NEW WORDS BEFORE WE USE THEM

Landing layer: Raw or near-raw source capture for traceability/replay. Staging layer: Typed/standardized/deduplicated intermediate data prepared for core modeling. Core layer: Conformed historical analytical facts/dimensions. Presentation layer: Curated marts/models shaped for consumer/business use.

DWF8 - Follow with me once

Page B - Order CSV to executive KPI

Order CSV to executive KPI

1. Landing stores immutable/near-raw files with source filename/arrival timestamp.
2. Staging standardizes data types/statuses, identifies duplicates and records rejected rows.
3. Core resolves conformed dimensions/SCD history and loads facts at declared grain.
4. Presentation creates business marts/semantic-facing structures; reports do not depend on raw source quirks.
5. Write a quality/ownership/replay contract for each layer and identify the earliest trustworthy layer for reprocessing after a business-rule change.

EXPECTED RESULT

A layered architecture where responsibilities and quality gates are distinct rather than four synonyms for folders.

WHY IT WORKED

Source volatility is absorbed before consumer-facing data and each stage can be independently validated.

DWF8 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: map customer API + transaction files to Landing -> Staging -> Core -> Presentation with one failure/replay route.

COMMON BEGINNER MISTAKES

1. Bronze/Silver/Gold names used without contracts. | 2. Landing cleaned so aggressively raw evidence disappears. | 3. Presentation layer becomes another uncontrolled transformation jungle. | 4. Duplicate business logic repeated in every report.

STUCK? RECOVERY

For each layer answer: What enters? What one row means? What changes are allowed? What quality is guaranteed? Who owns it? Who consumes it? Save the genuine attempt before review.

VALIDATE

Every layer has input/output grain, allowed transformation, quality checks and owner/consumer.

DWF8 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Fabric's medallion architecture uses Bronze/Silver/Gold for lakehouse organization; that is a current implementation pattern comparable to, but not identical with, every warehouse's Landing/Staging/Core/Presentation design.

TEACH IT BACK

Explain why layer names matter less than responsibilities/contracts.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry maps a schema drift incident through the layers.

EVIDENCE / MASTERY

Layer contract table + flow/replay diagram + quality/owner map + explain-back.

VISUAL/SOURCE ROUTE: Required visual support: current Microsoft medallion architecture article/diagram for Bronze -> Silver -> Gold.

https://learn.microsoft.com/en-us/fabric/onelake/onelake-medallion-lakehouse-architecture

DWF9 - Data lakes and lakehouses

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

Choosing Lakehouse because it is newer—or Warehouse because it is familiar—is architecture by fashion. The right store depends on data types, development tools, transaction/query needs and consumers.

WHAT YOU LEARN

Explain data lake versus lakehouse versus warehouse; choose a store from workload characteristics; design a combined lakehouse + warehouse pattern when roles are clear.

PREREQUISITE

DWF3 warehouse, DWF8 layering and basic file/SQL concepts.

NEW WORDS BEFORE WE USE THEM

Data lake: Scalable file/object storage for diverse data, often with flexible structure. Lakehouse: Architecture/platform combining lake storage with managed table/transaction/query capabilities. Delta Lake: Open table format/transaction-log approach widely used for lakehouse tables; Fabric uses Delta in OneLake.

DWF9 - Follow with me once

Page B - Choose for mixed analytics

Choose for mixed analytics

1. Structured financial reporting with T-SQL, governed star schemas and multi-table transactions strongly fits a Warehouse-style workload.
2. Large semi/unstructured data plus Spark/Python engineering and ML strongly fits a Lakehouse-style workload.
3. Mixed company pattern: raw/event files land/transform in Lakehouse, curated dimensional BI tables live in Warehouse, both serving Power BI where appropriate.
4. Do not duplicate data without need; evaluate platform features such as shared/open storage/shortcuts when available.
5. Record why each component exists. If two stores are used but no workload boundary is clear, simplify.

EXPECTED RESULT

A workload-based warehouse/lakehouse decision that can justify a combined design without claiming one universal winner.

WHY IT WORKED

Storage/tooling choice follows data/query/engineering/governance requirements.

DWF9 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: insurer has relational policies/claims plus PDFs/logs and wants BI + ML. Design a storage split and consumer paths.

COMMON BEGINNER MISTAKES

1. Lakehouse is always future-proof/better. | 2. Warehouse cannot coexist with lake storage. | 3. Using Spark because 'big data' sounds advanced on small structured BI. | 4. Choosing from job-title prestige rather than workload.

STUCK? RECOVERY

Score structuredness, preferred query engine, transactions, engineering/ML needs, BI pattern, governance and latency before choosing. Save the genuine attempt before review.

VALIDATE

Each data type/workload maps to a store/tool for a concrete reason; duplicate copies have a justification.

DWF9 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Current Fabric guidance differentiates Warehouse for structured enterprise SQL/BI and Lakehouse for broader data engineering/unstructured scenarios; both use OneLake/open table storage and can be combined.

TEACH IT BACK

Explain warehouse versus lakehouse as complementary workload choices, not old versus new technology.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses a retailer with mostly structured SQL BI plus clickstream JSON.

EVIDENCE / MASTERY

Decision matrix + architecture + data/tool/consumer mapping + explain-back.

VISUAL/SOURCE ROUTE: Microsoft Learn lakehouse overview is the required current visual/reference; Fabric decision guide is recommended comparison.

https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-overview

DWF10 - Batch vs streaming concepts

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

A 'real-time dashboard' can cost far more to build/operate while changing no decision. Streaming also adds late events, state, retry/replay and monitoring complexity.

WHAT YOU LEARN

Choose batch, micro-batch or streaming based on business decision latency, source event behavior, failure/recovery, cost and operational complexity; avoid real-time prestige.

PREREQUISITE

DWF7-DWF9 data-flow/storage concepts.

NEW WORDS BEFORE WE USE THEM

Batch: Process accumulated data at intervals. Streaming: Continuously process events/records as they arrive or in very short windows. Micro-batch: Small frequent batches that provide near-real-time freshness without fully continuous processing. Event time: When the business event happened, which can differ from processing/arrival time.

DWF10 - Follow with me once

Page B - Three business decisions

Three business decisions

1. Nightly finance close: batch is usually appropriate because the decision/report is daily and reconciliation matters more than seconds.
2. Hourly operations dashboard: micro-batch may meet the actual decision latency with simpler recovery/cost.
3. Fraud alert during payment authorization: streaming/near-real-time can be justified because delay directly reduces business value.
4. For streaming, discuss late/out-of-order events, checkpoint/state, retries and replay—not only faster refresh.
5. Document the maximum useful latency first, then choose architecture. Ask whether downstream users/actions can actually consume real-time updates.

EXPECTED RESULT

A latency decision table tied to business value and operational trade-offs.

WHY IT WORKED

Technology choice is made after defining how fast the decision must react.

DWF10 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: payroll, delivery tracking, IoT machine alert and executive revenue dashboard. Choose batch/micro-batch/streaming and justify.

COMMON BEGINNER MISTAKES

1. Streaming = modern, batch = legacy. | 2. Realtime because stakeholder said 'live' without decision threshold. | 3. Ignoring late events/replay/state. | 4. Continuous ingestion while downstream report refreshes only daily.

STUCK? RECOVERY

Ask: What action changes if data is 5 sec/5 min/1 hr/1 day old? Choose the slowest architecture that still preserves required value/reliability. Save the genuine attempt before review.

VALIDATE

Each choice includes maximum useful latency, failure/replay expectation, cost/complexity and downstream capability.

DWF10 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Hybrid architectures are common. Streaming may serve operational detection while authoritative financial/reconciled reporting remains batch.

TEACH IT BACK

Explain why an hourly micro-batch can be better engineering than streaming for some 'real-time' dashboards.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry uses stock replenishment and customer-notification use cases.

EVIDENCE / MASTERY

Latency decision matrix + failure/replay notes + explain-back.

VISUAL/SOURCE ROUTE: Recommended visual: Harsha Guggilla 'Batch vs Streaming Data Pipelines Explained' - 00:39 example, 07:12 batch, 20:31 streaming, 29:18

https://www.youtube.com/watch?v=fJEadN7_U6g

DWF11 - Data lineage and metadata

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

When an executive KPI is wrong or stale, 'it comes from Power BI' is not an answer. You need to trace the value back through transformations and know who owns each layer.

WHAT YOU LEARN

Distinguish technical/business metadata; draw end-to-end lineage across tools; record owner, definition, grain, refresh/freshness and classification; use platform lineage views as support, not the only documentation.

PREREQUISITE

DWF3-DWF9 architecture layers and semantic-model familiarity.

NEW WORDS BEFORE WE USE THEM

Lineage: Trace of where data came from, how it changed and where it is consumed. Metadata: Data about data: structure, definitions, owners, refresh, classification, technical properties. Freshness SLA: Agreed expectation for how current a data product must be.

DWF11 - Follow with me once

Page B - Trace Executive Revenue KPI

Trace Executive Revenue KPI

1. Source: Orders.LineAmount/Discount plus source owner and source timestamp.
2. Staging: typed/cleaned sales.net_sales with transformation rule/version.
3. Core: FactSales.NetSales at order-line grain and DimDate/Customer/Product relationships.
4. Semantic: [Total Sales] measure and report filter assumptions.
5. Executive KPI: display definition, owner, expected refresh/freshness and downstream decision. Link each step rather than stopping at the warehouse.

EXPECTED RESULT

End-to-end lineage that crosses source, transformation, warehouse, semantic/report and ownership/freshness.

WHY IT WORKED

Technical origin and business meaning are connected, making impact analysis and incident diagnosis possible.

DWF11 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: trace 'On-Time Delivery Rate' from shipment source through warehouse and executive dashboard.

COMMON BEGINNER MISTAKES

1. Lineage ends at table name. | 2. Business definition documented but technical source unknown. | 3. Owner/freshness omitted. | 4. Assume automated lineage captures every custom transformation/business rule.

STUCK? RECOVERY

Pick one KPI and walk backwards until the original source field. At each hop write transform, grain, owner and freshness. Save the genuine attempt before review.

VALIDATE

A reviewer can answer 'what breaks if this source column changes?' and 'who owns the stale stage?' from your lineage.

DWF11 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Fabric includes workspace lineage view, but organizations still need business definitions, owners and transformation context that automated graphs may not fully express.

TEACH IT BACK

Explain the difference between lineage and a data dictionary.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry performs impact analysis for renamed Product Category.

EVIDENCE / MASTERY

End-to-end lineage map + metadata register + impact/owner/freshness notes + explain-back.

VISUAL/SOURCE ROUTE: Current Microsoft Fabric lineage view is recommended visual support.

https://learn.microsoft.com/en-us/fabric/governance/lineage

DWF12 - Quality controls across a pipeline

Page A - why, outcome, prerequisite, mental model and vocabulary

WHY IT MATTERS

If you only check the final dashboard total, you know something is wrong but not where it entered. Reliable analytical platforms validate freshness, completeness, uniqueness, validity, referential integrity, history and business reconciliation at the layer where each risk occurs.

WHAT YOU LEARN

Design multi-layer quality checks with check/rule/observed/status/severity/owner/action; decide warning vs publication blocker; trace a failure to its first broken layer.

PREREQUISITE

All DWF1-DWF11, especially SCD, layers, lineage and grain.

NEW WORDS BEFORE WE USE THEM

Completeness: Whether expected data/records/files arrived. Referential integrity: Whether fact foreign keys resolve to valid dimension rows under the intended rules. Severity: Business/operational impact level determining warning, quarantine or release block.

DWF12 - Follow with me once

Page B - Quality plan from ingestion to report

Quality plan from ingestion to report

1. Ingestion: expected files/events arrived, schema matches, rows > 0, freshness within SLA.
2. Staging: types valid, required business keys present, duplicate rules applied, rejected rows counted.
3. Core/history: dimension surrogate keys unique, exactly one current SCD row, no effective-range overlap, fact foreign keys resolve.
4. Presentation: revenue/order counts reconcile to approved core controls; semantic/report freshness passes.
5. Every check stores name, rule/expected, observed, status, severity, timestamp, owner and action. Define which FAIL values block publication.

EXPECTED RESULT

A layered quality plan where a failed check points to an owner/action and release behavior.

WHY IT WORKED

Checks are placed close to the risks they detect, so root cause is localized before consumers see bad data.

DWF12 - Now do it yourself

Page C - fresh transfer, mistakes, recovery and validation

SMALL INDEPENDENT TASK

Fresh case: create 10 quality checks for a Sales + Customer SCD2 pipeline, including one warning and four publication blockers.

COMMON BEGINNER MISTAKES

1. Only grand-total reconciliation. | 2. Every check same severity. | 3. FAIL displayed but report still publishes. | 4. Quality result has PASS/FAIL but no observed/rule/owner.

STUCK? RECOVERY

List risks by layer: arrival, schema/type, key/duplicate, history/FK, business total, freshness. Add one repeatable check/owner/action per risk. Save the genuine attempt before review.

VALIDATE

Every check has rule, observed field, severity, owner/action and layer; injected duplicate/history/FK/freshness failures route correctly.

DWF12 - Prove mastery

Page D - professional version, teach-back, protected review and evidence

THEN SHOW THE PROFESSIONAL VERSION

Quality is operational behavior, not a checklist document. Controls should run repeatedly and feed monitoring/release decisions in later engineering systems.

TEACH IT BACK

Explain why one final dashboard total cannot tell you whether a file was late, duplicated or historically mis-keyed.

PROTECTED REVIEW + CHANGED RETRY

Attempt first. Changed retry injects a late file plus overlapping SCD2 rows.

EVIDENCE / MASTERY

Quality-control register + severity/release policy + failure-routing exercise + explain-back.

VISUAL/SOURCE ROUTE: Book-led; DWF8/DWF11 current platform visuals provide context.

https://learn.microsoft.com/en-us/training/paths/work-with-data-warehouses-using-microsoft-fabric/

C2-06 mastery checklist + quick reference

Use after the first pass; architecture is proved by changed-case design

TWELVE QUESTIONS

DWF1 Is this workload running or understanding the business? | DWF2 What transaction/scan/history pattern drives OLTP/OLAP? | DWF3 What does the warehouse govern beyond storage? | DWF4 What does one row in every fact mean? | DWF5 Which key identifies entity vs warehouse version? | DWF6 Should this attribute overwrite or preserve history? | DWF7 Where does transformation run and from where can we replay? | DWF8 What contract does each layer guarantee? | DWF9 Why Warehouse/Lakehouse/both for this workload? | DWF10 How fast must the decision react? | DWF11 Can we trace KPI to source + owner/freshness? | DWF12 Which stage catches each failure and does it block release?

BLOCK MASTERY IF

Fact grain mixed; target/snapshot multiplied; business/surrogate key roles confused; SCD2 overlaps/two current rows; fact resolves to wrong historical version; layer has no contract/replay path; storage chosen by fashion; real-time chosen without value; lineage stops early; quality failure has no severity/owner/release action.

FAST ARCHITECTURE DEBUG

Business process -> row grain -> keys/history -> source/freshness -> layers/transforms -> store/tool -> latency -> lineage -> quality/release. Find the first broken contract instead of redrawing the entire platform.

OPTIONAL PATH / DE BRIDGE

C2-06 is optional Senior/BI depth but intentionally has Very High DE overlap. Direct learners get the required bridge version in C2B, so C2A completion still cannot block C2B/C3.

C2-06 source / video registry

Durable concepts first; current Fabric references only where they clarify implementation

DWF1-3 - Microsoft Learn - Get started with data warehouses in Fabric
https://learn.microsoft.com/en-us/training/modules/get-started-data-warehouse/
7m warehouse concepts; 8m Fabric Warehouse; 7m modeling

DWF3-4 - Fabric Data Warehouse / dimensional modeling
https://learn.microsoft.com/en-us/fabric/data-warehouse/dimensional-modeling-overview
Star-schema/warehouse implementation reference

DWF5-6 - Fabric dimension-table guidance
https://learn.microsoft.com/en-us/fabric/data-warehouse/dimensional-modeling-dimension-tables
Keys/SCD warehouse reference

DWF6 - Fabric SCD Type 2
https://learn.microsoft.com/en-us/fabric/data-factory/slowly-changing-dimension-type-two
Type 2 version-row implementation example

DWF6 - Fabric SCD Type 1
https://learn.microsoft.com/en-us/fabric/data-factory/slowly-changing-dimension-type-one
Type 1 overwrite contrast

DWF7 - Fabric warehouse learning path
https://learn.microsoft.com/en-us/training/paths/work-with-data-warehouses-using-microsoft-fabric/
Load strategies, pipelines, full/incremental/SCD context

DWF8 - Fabric medallion architecture
https://learn.microsoft.com/en-us/fabric/onelake/onelake-medallion-lakehouse-architecture
Current Bronze/Silver/Gold diagram; recommended Fabric pattern

DWF9 - Fabric lakehouse overview
https://learn.microsoft.com/en-us/fabric/data-engineering/lakehouse-overview
Warehouse vs Lakehouse current capability context

DWF9 - Fabric data-store decision guide
https://learn.microsoft.com/en-us/fabric/fundamentals/decision-guide-data-store
Current workload/store decision guidance

DWF10 - Harsha Guggilla - Batch vs Streaming Data Pipelines
https://www.youtube.com/watch?v=fJEadN7_U6g
00:39 example; 07:12 batch; 20:31 streaming; 29:18 decision; 36:40 trade-offs

DWF11 - Fabric lineage
https://learn.microsoft.com/en-us/fabric/governance/lineage
Current workspace/item lineage view

DWF12 - Fabric warehouse learning path
https://learn.microsoft.com/en-us/training/paths/work-with-data-warehouses-using-microsoft-fabric/
Monitoring/load quality context; course controls remain internal

VERIFY / FALLBACK

Current Fabric references and the batch-vs-streaming video metadata were rechecked 2026-09-10. External media is not required to pass; the internal architecture book is authoritative for assessed concepts.